ASRP: Adaptive Support-Based Response Prediction for Test-Time AI Capability Evaluation

Yuhan Chiu
National Taiwan University

Abstract

Predicting whether an AI system will solve a particular evaluation item requires accounting for item difficulty, subject ability, and the small number of revealed labels. We present **ASRP (Adaptive Support-Based Response Prediction)**, a method for predicting subject–item response probabilities in the Predictive AI Evaluation Competition. ASRP combines a public-data subject and metadata prior, a MiniLM-based amortized Rasch predictor, and a support-conditioned kernel probe. A budget-dependent ensemble combines these branches, followed by conservative topic residual calibration; source-keyed guards are available when a public-source identifier matches. Its streaming acquisition policy uses an adaptive covariance-gain rule with a bounded warm-up phase. We evaluate 92 public-source groups over three fixed seeds, with 22,357 evaluation outcomes per seed. A paired comparison of ASRP and Empirical Mean under both Random and Adaptive acquisition separates predictor quality from acquisition effects. ASRP + Adaptive achieves a mean Brier ALC of 0.1987, compared with 0.2008 for ASRP + Random and 0.2435 for Empirical Mean + Random (lower is better).

1 Introduction

Modern AI evaluations produce response tables with a binary outcome for each subject–item pair. A benchmark average is useful for ranking systems, but it does not provide a probability for a new item and it can be unstable when only a small subset of items is queried. The Predictive AI Evaluation Competition (PAIEC) therefore asks participants to predict the probability that a subject answers each item correctly while revealing at most 31 labels per subject–benchmark pair.

Our goal is a predictor that uses item text and visible metadata at label budget zero, then adapts to revealed labels without overreacting to a small or unrepresentative support set. We implement **ASRP** as a competition submission rather than as a new psychometric theory.

Relation to item-response modeling. Lord's treatment of item response theory (IRT) models the probability of a correct response as a function of test-taker ability and item difficulty [Lord, 1980]. This is the basis for ASRP's Rasch-style probability: a public subject prior supplies the initial ability, the frozen text model supplies item difficulty, and revealed labels update ability with a penalized Newton–Raphson step.

Content-based and adaptive evaluation. Reliable and Efficient Amortized Model-based Evaluation shows that benchmark averages can be distorted by question difficulty, and proposes amortizing difficulty from question content together with adaptive item selection [Truong et al., 2025]. ASRP uses the same two ideas in the competition setting: MiniLM features predict unseen-item difficulty, while `adaptive_4_warm` implements support selection when the remaining budget permits discretionary choices. ASRP does not use that paper's question generator or evaluation model.

Preprint.

Task-level capability prediction. Agent Psychometrics extends IRT to predict task-level success for agentic coding benchmarks using task features and separate ability factors [Ge et al., 2026]. ASRP follows the same subject–item perspective, but uses the competition's visible metadata, binary outcomes, and a support-conditioned kernel residual instead of an agent/scaffold decomposition.

Capability calibration. *On Calibration of Large Language Models: From Response to Capability* distinguishes confidence that one sampled response is correct from confidence that a model can solve a query overall [Yang et al., 2026]. ASRP targets the latter quantity: $p(x)$ is the probability that the subject answers the current item correctly, and Brier score evaluates this query-level probability. This motivates our query-level prediction target; ASRP uses its own topic residual correction rather than reproducing that paper's confidence estimators.

ASRP combines frozen text-based difficulty and public subject priors with a kernel residual fitted to acquired labels. Fixed budget-specific weights blend these signals, followed by shrunk topic corrections.

We compare against the official Empirical Mean on a local public proxy. Competition-specific artifacts use permitted public measurement-db data; runtime inference never queries hidden responses.

2 Competition setting

At runtime the platform supplies a pair of dictionaries $x = (s, i)$ describing a subject and an item, together with a list of previously acquired binary observations. The predictor returns a finite probability in $[0, 1]$. Eligible subject–benchmark groups have at least 80 distinct items. The platform creates a fixed acquisition/evaluation split and evaluates the same evaluation items at the **six label budgets**

$$n \in \{0, 1, 3, 7, 15, 31\}.$$

Acquisition items are never scored and evaluation outcomes are never revealed. The optional acquisition hook receives the current candidate, acquired labels, and remaining-label context; the interface can also supply a current prediction when requested. ASRP uses the first three inputs.

For a budget n and a subject–benchmark group with N_{eval} evaluated responses, the Brier score is the mean squared error between the predicted probability p and the binary outcome y:

$$B_n = \frac{1}{N_{\text{eval}}} \sum_{j=1}^{N_{\text{eval}}} (p_j - y_j)^2.$$

The group-level Brier ALC is the normalized area on the logarithmic budget axis and is equivalent to

$$\text{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The official score averages these group-level ALC values equally across subject–benchmark groups. **Lower is better.** All reported local tables use this same weighting.

3 Data and permitted training

The predictor uses frozen artifacts trained and calibrated from the public `aims-foundations/measurement-db` release [AIMS Foundations, 2026]. The evaluation starts from a fixed inventory of 142 binary sources and retains sources containing a complete subject group with 80–1,000 distinct binary-response items. This yields **92 sources**; 50 sources have no qualifying group. One eligible subject is fixed per source, and all items in that group are retained without truncation. The selected groups contain 44,741 items in total. Appendix B lists the evaluated sources; the manifest records both inclusions and exclusions. Upstream metadata and licenses remain authoritative.

The subject prior and metadata statistics are frozen public-response aggregates. Their runtime use and fallback rules are specified in Appendix A; reproducing the reported scores uses the released weights without refitting them. MiniLM is used as a pretrained encoder; the packaged encoder is `sentence-transformers/all-MiniLM-L6-v2`, based on MiniLM [Wang et al., 2020]. Its weights are **frozen at runtime**. The difficulty model, priors, kernel configuration, calibration statistics, and cluster centroids are prepared offline and bundled with the submission; kernel residual

coefficients are fitted online from revealed labels. The current submission uses **no external LLM or web search API during scoring**. At runtime, the program uses only the supplied subject and item attributes, anonymous benchmark identifiers, and labels revealed within the allowed budget; it does not access hidden evaluation outcomes.

4 ASRP method

Figure 1 gives the complete data flow. The upper lane contains offline artifacts that are fixed before evaluation. The lower lane shows one runtime prediction and the separate label-acquisition loop. Solid arrows show the prediction and acquisition paths; dashed arrows show frozen artifacts and revealed evidence supplied to these paths.

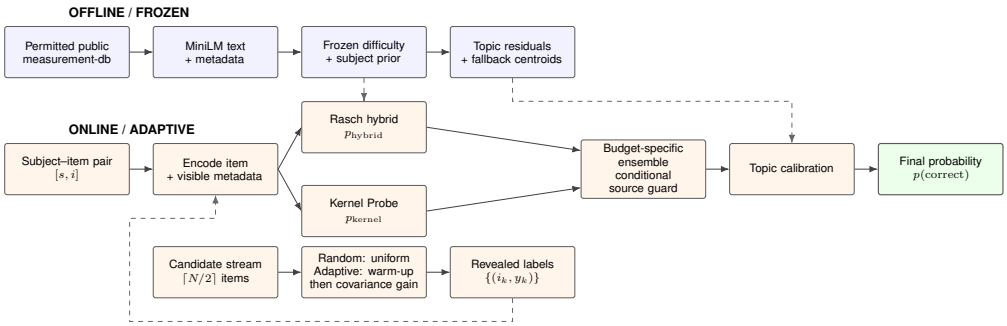


Figure 1: ASRP overview. Frozen public-data artifacts supply difficulty, prior, and calibration estimates. Revealed labels adapt the Rasch hybrid and kernel probe before budget-specific ensembling and topic correction. The source guard is inactive for anonymous proxy identifiers. Candidate pools contain half of each complete group. Random samples uniformly; Adaptive skips up to 16 warm-up candidates before covariance-gain selection. Both supply nested label prefixes.

4.1 Item representation and amortized difficulty

Let $e_i \in \mathbb{R}^{384}$ be the sentence embedding of the normalized item text from MiniLM. We concatenate e_i with six deterministic numeric text features $z_i \in \mathbb{R}^6$ (including text length, digit proportion, and code or mathematical markers). The **frozen difficulty network** standardizes this vector and applies a three-layer feed-forward map (390–128–32–1):

$$d_i = f_\phi([e_i; z_i]).$$

To compute e_i, the whitespace-normalized text is restricted to its first 6,000 characters and tokenized to at most 256 tokens. Mean pooling and L_2 normalization yield e_i. The hidden layers use ReLU activations; standardized inputs are clipped to $[-10, 10]$ and the resulting difficulty to $[-2, 2]$. This content-based estimate is an amortized substitute for fitting a separate difficulty parameter for every unseen item.

4.2 Public prior and Rasch update

The hybrid uses two public-data priors. The subject-only prior initializes ability, while the metadata prior p_{src} combines the subject's public response rate with visible item-feature statistics for evidence smoothing. For the frozen public subject offset o_s keyed by normalized model name (zero for an unknown name), define

$$m_s = \text{clip}(0.5 + 0.5o_s, 0.05, 0.95), \qquad \theta_0 = \text{logit}(m_s).$$

The Rasch probability is

$$p_{\text{Rasch}}(i) = \sigma(\theta - d_i), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Here d_i is the frozen content-based item difficulty from Section 4.1. For acquired labels $(y_k, d_k)_{k=1}^n$, ASRP obtains a **maximum a posteriori (MAP) estimate** by maximizing the Bernoulli log-likelihood

with a Gaussian prior centered at θ_0:

$$\ell(\theta) = \sum_{k=1}^{n} \log p(y_k \mid \theta, d_k) - \frac{(\theta - \theta_0)^2}{2v}, \qquad v = 2.0.$$

The implementation uses Newton–Raphson iteration with clipped steps. Writing $p_k = \sigma(\theta - d_k)$,

$$g = \sum_k (y_k - p_k) - \frac{\theta - \theta_0}{v}, \qquad h = \sum_k p_k(1 - p_k) + \frac{1}{v}, \qquad \theta \leftarrow \theta + \mathrm{clip}(g/h, -2, 2).$$

It stops when the step magnitude is below 10^{-7} or after 30 iterations, and clips the final ability to $[-12, 12]$. At budget zero, $\theta = \theta_0$.

The metadata prior supplies a smoothed evidence estimate,

$$p_{\mathrm{smooth}} = \frac{S_n + \kappa p_{\mathrm{src}}}{n + \kappa}, \qquad S_n = \sum_k y_k, \quad \kappa = 4.$$

The Rasch and smoothed terms are combined as

$$p_{\mathrm{hybrid}} = (1 - \lambda_n) p_{\mathrm{smooth}} + \lambda_n p_{\mathrm{Rasch}}.$$

The metadata prior uses the count-weighted lookup in Appendix A. In the released configuration, metadata-match adaptation is disabled and the prior shift is zero. The frozen weights are $\lambda_n = (0.2875, 0.3000, 0.2875, 0.2750, 0.2250, 0.2375)$ for budgets $(0, 1, 3, 7, 15, 31)$.

4.3 Support-conditioned kernel probe

The **support-conditioned kernel probe** estimates a local correction from the revealed support items. The implementation supports revealed responses from multiple subjects in the current benchmark; this proxy supplies only the current subject's labels. For each support item r, let u_r be its MiniLM vector and v_r its hashed lexical vector, both derived from visible item text, metadata, and interactor descriptions. Unlike the Rasch text encoder, the kernel encoder pools up to four evenly spaced 254-token windows, normalizes each window vector, and normalizes their sum. The lexical vector has 8,192 hashed coordinates. Support-only centering, variance scaling, and lexical inverse-frequency weighting give normalized vectors $\tilde{u}_r, \tilde{v}_r$; the query uses the same fitted transformation (Appendix A). For any pair of support items r, t, the similarity kernel is

$$q_{rt} = \frac{\tilde{u}_r^\top \tilde{u}_t + \tilde{v}_r^\top \tilde{v}_t}{2}, \qquad K_{rt} = \exp\left(\frac{\min(q_{rt} - 1, 0)}{\tau}\right),$$

with $\tau = 4 \max(d_{\mathrm{med}}, 10^{-5})$, where d_{med} is the median off-diagonal distance $1 - q_{rt}$ above 10^{-8}, or 0.1 if none qualifies. The query-to-support vector $k(x)$ uses the same transformation and kernel.

The kernel intercept is a two-component Beta mixture. For subject a, let s_a of n_a revealed labels be successes and $m_a = \mathrm{clip}(0.5 + 0.5 o_a, 0.05, 0.95)$. Concentrations $c \in \{1, 8\}$ have prior masses $(0.3, 0.7)$, updated by the Beta–Bernoulli marginal likelihood to normalized weights ω_{ac}:

$$b_a = \sum_c \omega_{ac} \frac{m_a c + s_a}{c + n_a}.$$

Appendix A gives the weight update. This subject baseline is combined with a local residual fitted to the revealed labels.

Let M average rows belonging to the same subject, and let $C = I - M$. The centered ridge residual coefficients are

$$\gamma = C(CKC + \lambda I)^{-1} C y, \qquad \lambda = 0.3,$$

where y contains the revealed binary labels. If $\bar{f}_a$ is the subject mean of the fitted kernel contribution, the query prediction is

$$p_{\mathrm{kernel}}(x, a) = \mathrm{clip}\big(b_a + k(x)^\top \gamma - \bar{f}_a, 0, 1\big).$$

The kernel contribution is suppressed at budgets 0 and 1 and receives weights $(0, 0, 0.2, 0.4, 1, 1)$ at $(0, 1, 3, 7, 15, 31)$, so a tiny support set cannot override the cold-start prediction. This creates a

deliberate budget-dependent division of labor: at low budgets, the Rasch hybrid is the main predictor because its public subject prior and content-based difficulty remain stable when there are too few revealed labels to fit a local kernel residual. The kernel branch is introduced gradually at 3 and 7 labels, then becomes the nominal primary predictor at 15 and 31 labels, giving greater influence to residual corrections fitted from the larger support set. The late-budget source guard can reduce this kernel weight for sources with a regression identified in public diagnostics.

4.4 Topic calibration and source guard

The calibration artifact is generated from public binary responses. ASRP scans the visible metadata in its original order and uses the first topic key–value pair present in the frozen topic-residual table. A topic residual $r_{t,n}$ is shrunk toward zero using its public count m_t:

$$\Delta_{t,n} = \mathrm{clip}\left(\frac{m_t}{m_t + 40} r_{t,n}, -0.06, 0.06\right).$$

If no topic key–value pair matches the table, the Rasch encoder's 384-dimensional vector is assigned to the closest of eight frozen centroids by cosine similarity. Cluster residuals use a stronger prior count of 200 and a smaller cap of 0.03. Writing w_n for the kernel weights in Section 4.3 and $g_{\mathrm{src},n}$ for the source guard below (zero for $n < 15$), the complete prediction is

$$\tilde{w}_n = w_n(1 - g_{\mathrm{src},n}), \qquad p_{\mathrm{final}} = \mathrm{clip}((1 - \tilde{w}_n)p_{\mathrm{hybrid}} + \tilde{w}_n p_{\mathrm{kernel}} + \Delta_{t,n}, 0, 1)\,.$$

The correction is zero if neither a topic nor a fallback cluster has stored statistics.

Public validation identified some sources for which the kernel branch became less accurate at high budgets. At budgets 15 and 31, a matching public-source key supplies a guard $g \in [0, 0.35]$ that multiplies the nominal kernel weight by $1 - g$; the remaining weight goes to the Rasch hybrid. Unmatched identifiers receive zero adjustment. The anonymous aliases in the reproducible proxy do not match these source-name keys, so this guard is inactive in the reported experiment. This guard is a conservative regularizer, not a lookup of hidden test outcomes; source statistics are learned offline from public data.

4.5 Adaptive acquisition

The submitted `labeling.py` implements `adaptive_4_warm`. It skips up to 16 candidates to establish feature geometry; skipping reveals no label and consumes no label budget. A rolling reference set P deduplicates the last 64 seen candidates by their visible text/metadata key. Conditional covariance gain a_j measures how much observing candidate j could reduce uncertainty on P (Appendix A.3). Define

$$u_j = \min\left(\frac{a_j}{|P|^{-1}\sum_{r\in P} a_r}, 20\right)^4, \qquad \pi_c = \min\left(1, \frac{L}{R}\,\mathrm{clip}\left(\frac{u_c}{|P|^{-1}\sum_{r\in P} u_r}, 0.05, 20\right)\right),$$

where c is the current candidate, L the remaining label budget, and R the remaining candidates including c. The hook acquires c when its deterministic uniform hash is below π_c. It acquires every remaining candidate when $R \le L$, overriding warm-up. Gain depends on supplied item features and the acquired support, not on unseen outcomes or future candidates. The exponent four sharpens the preference for high-gain candidates; it does not denote four warm-up items.

5 Experimental setup

5.1 Public proxy protocol

Complete groups and repeated splits. The local protocol retains complete groups with $80 \le N \le 1{,}000$ distinct items; the upper bound is a local eligibility filter, not truncation. Within each eligible source, SHA256 ordering with the fixed group-selection seed 20261028 selects one subject, identically across all experiments. The actual group sizes range from 90 to 1000. For repeated responses to an item, the smallest trial number and then response ID select one binary outcome, held fixed across policies and seeds. Splitting uses seeds **42, 43, and 44**: a seeded SHA256 ordering assigns the first $\lceil N/2 \rceil$ items to acquisition and the rest to evaluation. Thus candidate pools contain 45–500 items, and each seed scores **22,357 evaluation outcomes**. The dataset revision, table checksums, subject IDs, response IDs, and three splits are fixed in the released manifest.

Paired predictor and acquisition comparison. We evaluate **ASRP and Empirical Mean under both Random and Adaptive acquisition**. Within a seed, both policies see the same candidate order and evaluation pool. Random acquisition accepts the current candidate with probability L/R, where L is the remaining label budget and R counts remaining candidates including the current one. Its random variates are deterministic hashes of the visible input and context, with seed-specific anonymous subject aliases. Adaptive acquisition executes the packaged `adaptive_4_warm` hook. Its warm-up is $\min(16, A - 31)$ for a candidate pool of size A; remaining candidates are mandatory when their count equals the remaining budget. Both predictors receive identical acquired labels *within each policy*, using nested prefixes at budgets $0, 1, 3, 7, 15, 31$. The two policies can select different labels. The acquisition context targets 31 labels throughout; checkpoints do not restart selection. Every checkpoint scores the same evaluation items within that seed.

Evaluation scope. Evaluation outcomes are never supplied online, but this acquisition/evaluation split is not independent of all offline fitting: frozen public training and calibration use the same release. The local scores may therefore be optimistic. They are not Codabench scores; private data and formative random draws can produce different results.

5.2 Baseline and metrics

We use the **Empirical Mean predictor** from the official PAIEC repository [Truong et al., 2026]. For n acquired binary labels in the current subject–benchmark group, it predicts

$$p_{\mathrm{EM}} = \begin{cases} 0.5, & n = 0, \\ \frac{1}{n}\sum_{k=1}^{n} y_k, & n > 0. \end{cases}$$

It uses neither item features nor a learned prior. We run this unchanged predictor on each policy's acquired evidence. Empirical Mean + Random corresponds to its default acquisition choice; Empirical Mean + Adaptive isolates the effect of giving the same predictor ASRP's selected labels. The local Random policy reproduces uniform sampling without replacement, not the private server's particular random draw.

For each seed and policy, we compute Brier scores within each subject–source group and average groups equally, then average the three seed-level scores equally. Groups have different numbers of evaluation items, so pooling all response rows would give different weights and is not used. Brier ALC uses the weights in Section 2. We report the individual seed ALCs and their sample standard deviation; these describe variation across the three specified splits, not a confidence interval over all possible evaluations.

6 Results and analysis

6.1 Public proxy results

Table 1 reports the **four predictor–acquisition combinations**, averaged equally over groups and then over seeds 42, 43, and 44. ASRP + Adaptive has a mean Brier ALC of 0.1987, compared with 0.2435 for Empirical Mean + Random (an absolute reduction of 0.0447). Its aggregate Brier score is lower at 6 of the six budgets. The two predictors share identical labels within each acquisition policy; the Random and Adaptive policies use identical candidate orders and evaluation pools.

Predictor + acquisition	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
Empirical Mean + Random	0.2500	0.3488	0.2518	0.2086	0.1910	0.1842	0.2435
Empirical Mean + Adaptive	0.2500	0.3594	0.2366	0.2003	0.1883	0.1837	0.2403
ASRP + Random	0.2263	0.2166	0.2075	0.1947	0.1845	0.1755	0.2008
ASRP + Adaptive	0.2263	0.2161	0.2043	0.1912	0.1819	0.1738	0.1987

Table 1: Public proxy Brier scores averaged equally over 92 groups and seeds 42, 43, and 44. Lower is better. Evaluation pools are matched across policies; acquired labels are matched across predictors within each policy.

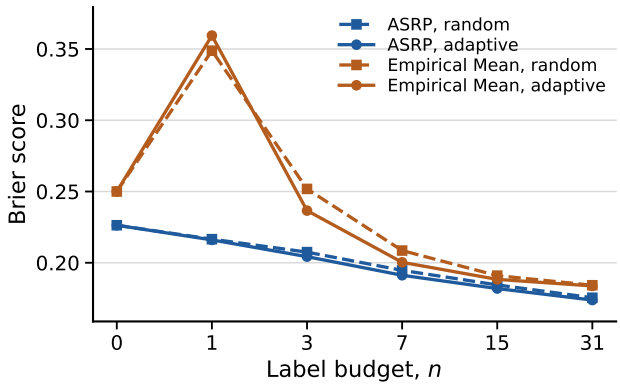


Figure 2: Mean public-proxy Brier curves for the four predictor–acquisition combinations on the official $\log_2(1+n)$ budget axis. Solid lines denote Adaptive acquisition and dashed lines Random; values match Table 1.

Predictor + acquisition	Seed 42	Seed 43	Seed 44	SD
Empirical Mean + Random	0.2467	0.2401	0.2435	0.0033
Empirical Mean + Adaptive	0.2386	0.2427	0.2396	0.0021
ASRP + Random	0.2020	0.1984	0.2020	0.0021
ASRP + Adaptive	0.1983	0.1983	0.1995	0.0007

Table 2: Group-averaged Brier ALC for each split/acquisition seed. SD is the sample standard deviation over the three seed scores. Lower ALC is better.

6.2 Prediction and acquisition effects

Predictor effect with acquisition held fixed. Under Random acquisition, ASRP's mean ALC is 0.2008 versus 0.2435 for Empirical Mean, a reduction of 0.0426. Under Adaptive acquisition, ASRP's mean ALC is 0.1987 versus 0.2403 for Empirical Mean, a reduction of 0.0416. Empirical Mean + Random rises from 0.2500 at zero labels to 0.3488 at one label: a single outcome makes its prediction either zero or one. ASRP smooths early evidence using public priors. At budget zero the acquisition policies have no effect. The Rasch hybrid followed by topic correction supplies the initial prediction; the kernel branch receives increasing weight as labels become available.

Acquisition effect with the predictor held fixed. For ASRP, Adaptive lowers mean ALC by 0.0021 relative to Random, and has lower ALC in all three seeds. For Empirical Mean, Adaptive lowers mean ALC by 0.0031 relative to Random, and has lower ALC in 2 of the three seeds. The closest ASRP pair differs by only -0.00005 (Adaptive minus Random, seed 43). Table 2 shows the individual seed results and their sample standard deviation; three seeds summarize sensitivity to these draws rather than establishing universal superiority.

Selection capacity. Across the full experiment, 273 of 276 Adaptive trajectories execute at least one discretionary decision. The 3 trajectories without discretionary choices belong to one 90-item group: 14 warm-up skips leave exactly 31 candidates, all mandatory. Topic calibration is active; the source-name guard remains inactive because the benchmark aliases are anonymous.

7 Limitations

Sparse metadata and local adaptation. Some public items lack usable topic metadata, motivating the frozen embedding-cluster fallback. Topic groups with few observations can have residual estimates dominated by sampling noise. This motivates shrinking and capping topic corrections. Likewise, an unrepresentative acquired support set can cause the kernel to over-update. The source guard only

limits its weight when a public-source key matches; anonymous or unseen sources do not receive this adjustment. These considerations motivate conservative correction magnitudes; neither topic calibration nor local adaptation guarantees improvement on every source.

8 Reproducibility and disclosure

The release separates inference code in `src/asrp` from frozen artifacts in `artifacts/asrp`. The entry points are `model.py` and `labeling.py`. From the repository root, run:

```
uv sync --locked
uv run hf auth login
uv run --locked python tools/reproduce.py
```

After accepting upstream access conditions and authenticating with a Hugging Face read token, this command downloads and verifies the pinned tables, reconstructs all splits, builds the submission, and evaluates all four combinations at three seeds. It checks every prediction against the published run within 10^{-7} and requires identical acquired labels. Frozen weights are used directly without refitting or parameter selection. The CPU evaluator uses four workers and caches only deterministic item features. Two independent complete executions produced identical predictions and acquisition records.

`experiments/` records the protocol, checksums, and selected subject/item/response IDs. `results/proxy` contains predictions, acquired labels, decisions, and metrics. `tools/update_report_results.py` generates the reported numbers; task text is downloaded from upstream.

Kernel weights are in `src/asrp/model.py`; Rasch settings and calibration are in `artifacts/asrp/runtime_config.json` and `artifacts/asrp/topic_calibration.json`. Code, frozen weights, the locked CPU environment, and report source are available at `https://github.com/Hubert0916/arsp`. The official checker remains in the upstream starting kit.

The competition-specific artifacts were produced from the permitted public measurement-db inventory. We use the pretrained MiniLM encoder and cite its model family. Codex was used as a coding assistant during development; it is not used as an online predictor in the current submission. No API key is included in the codebase or report.

9 Conclusion

ASRP provides a practical, budget-aware predictor for subject–item capability evaluation. Its main engineering principle is to use content-based and psychometric structure at cold start, use support-conditioned residuals only when evidence is available, and shrink topic corrections and late-budget updates toward conservative defaults. The three-seed public proxy gives ASRP + Adaptive a mean Brier ALC of 0.1987. Compared with Random acquisition using the same predictor, Adaptive reduces mean ALC by 0.0021. The paired design separates changes in prediction quality from changes in which labels are acquired.

A Runtime details

These equations specify inference from the released artifacts. The reproduction command evaluates those frozen artifacts; it does not repeat their original offline fitting. Subject offsets are stored in `binary_subject_prior.json`, count tables in `feature_prior.json`, difficulty weights and standardization in `difficulty_model.npz`, and residual tables and centroids in `topic_calibration.json`.

A.1 Metadata prior and unknown inputs

Let $(S_\star, N_\star)$ be global public successes and counts, and (S_s, N_s) the corresponding subject totals keyed by `normalized_name`. Define

$$q_\star = S_\star/N_\star, \qquad q_s = (S_s + 8q_\star)/(N_s + 8).$$

An unknown subject uses global totals, giving $q_s = q_\star$. Separately, its ability offset is zero, so the Rasch ability prior is zero; the metadata prior need not be 0.5.

Visible metadata are parsed into normalized key–value tokens and selected key–word tokens by `_feature_tokens`. For each matching token t, let $q_t = S_t/N_t$. When subject–token counts exist, set

$$q_{st} = \frac{S_{st} + 8q_t}{N_{st} + 8}, \qquad \rho_{st} = \frac{\min(N_{st}, 80)}{\min(N_{st}, 80) + 12}.$$

Otherwise set $q_{st} = q_t$ and $\rho_{st} = \min(N_t, 300)/(\min(N_t, 300) + 100)$. Tokens without global statistics are ignored. The query-specific prior is

$$p_{\mathrm{src}} = \mathrm{clip}\left(\frac{2q_s + \sum_t \rho_{st} q_{st}}{2 + \sum_t \rho_{st}}, 0.03, 0.97\right).$$

With no matching tokens it reduces to clipped q_s. Exact token normalization and stop words are specified in `src/asrp/model.py`.

A.2 Feature geometry and Beta weights

The six Rasch numeric features are $\log(1 + \text{text length})/10$, digit fraction, non-ASCII fraction, and indicators for code, mathematical notation, and a question mark. Fractions and the question-mark indicator use the first 2,000 characters; code/math markers use the first 6,000. The kernel lexical vector uses signed CRC32 hashing of tokens and adjacent token bigrams into 8,192 bins, log-count weighting, and L_2 normalization.

For $m > 1$ support vectors, the semantic scale and lexical weights are

$$a_j = \left(\mathrm{Var}_r(u_{rj}) + \max(\mathrm{median}_j \mathrm{Var}_r(u_{rj}), 10^{-7})\right)^{-1/2}, \qquad b_\ell = \log\frac{1+m}{1+f_\ell} + 0.1,$$

where f_ℓ counts lexical coordinates with magnitude above 10^{-10}. Thus

$$\tilde{u}_r = \mathrm{norm}((u_r - \bar{u}) \odot a), \qquad \tilde{v}_r = \mathrm{norm}(v_r \odot b).$$

For $m = 1$, centering and coordinate reweighting are bypassed. Normalization denominators are floored at 10^{-12}. Query items do not alter this support-fitted geometry.

For the Beta mixture, $\alpha_{ac} = m_a c$, $\beta_{ac} = (1 - m_a)c$, and

$$\omega_{ac} \propto \pi_c \frac{\mathrm{B}(\alpha_{ac} + s_a, \beta_{ac} + n_a - s_a)}{\mathrm{B}(\alpha_{ac}, \beta_{ac})}, \qquad (\pi_1, \pi_8) = (0.3, 0.7), \qquad \sum_c \omega_{ac} = 1.$$

The implementation normalizes these weights in log space. At zero labels the mixture mean is m_a, although its ensemble weight is zero. In Section 4.3, $M_{rt} = 1/n_a$ when rows r, t belong to subject a and zero otherwise; $\bar{f}_a$ is the mean of $(K\gamma)_r$ over that subject's support rows.

A.3 Covariance calculation for Section 4.5

The gain a_j used in Section 4.5 is computed as follows.

Let S be the acquired support and U the concatenation of the rolling reference set P and current candidate c. Unlike prediction geometry, acquisition geometry uses features of S, P, and c, all already visible. Its RBF bandwidth is computed from pairs in P only, using the same median-distance rule as Section 4.3. Let K be the resulting kernel. For the current subject s, define

$$\begin{aligned} A_{SS}[r,t] &= K_{SS}[r,t] + 0.25\,\mathbf{1}[s_r = s_t], \\ A_{US}[j,r] &= K_{US}[j,r] + 0.25\,\mathbf{1}[s_r = s], \\ A_{UU} &= K_{UU} + 0.25\mathbf{1}\mathbf{1}^\top. \end{aligned}$$

A unit observation-noise term gives

$$\Sigma = A_{UU} - A_{US}(A_{SS} + I)^{-1} A_{US}^\top, \qquad a_j = \max\left(\frac{|P|^{-1}\sum_{r\in P}\Sigma_{rj}^2}{1 + \max(\Sigma_{jj}, 0)}, 10^{-12}\right).$$

Without support, $\Sigma = A_{UU}$. Utility-average denominators are floored at 10^{-12}; without past candidates the policy falls back to L/R. The same SHA256 construction supplies the Random and Adaptive policies' uniform variates from the current input/context. No answer is supplied for a skipped candidate. The main experiment has $2 \times 3 \times 92 = 552$ policy trajectories, each acquiring 31 labels; each evaluated item receives predictions from both models at all six checkpoints.

B Public source inventory

The following 92 public binary source identifiers occur in the paired three-seed evaluation. They identify the evaluation sources; the benchmark inventory and upstream licenses remain authoritative.

```
afrieval
agentic_review_perturbation
algotune
alignment_faking
appworld
averimatec
babilong
beavertails
benger
bird_sql
braveguard
bridging_gap
cellverse
critic_discernment_game
crow
devbench
editbench
edu_circuit_hw
egoschema
elicitation_game
engdesign
enginemt_qa
exploitgym
faithcot
fewshot_ttt_bbh
financebench
finegrain_t2i
hallusionbench
harmbench
healthadminbench
helm_bbq
helm_thaiexam
hle
igakuqa119
ikp
jailbreakbench
kernelbench
legal_rag_bench
llmail_inject
llms_lean_formalization
mathconstraint
mathvista_mini
medhelm_ehr_sql
medhelm_head_qa
medhelm_med_mcqa
medhelm_med_qa
medhelm_medbullets
medhelm_medcalc_bench
medhelm_medec
medhelm_medhallu
medhelm_pubmed_qa
medhelm_race_based_med
mj_bench
mmbench_v11
monkey_power_laws
morphkv
mtbbench
multi_swebench
naep_llm_students
naturebench
nester
nyu_ctf_bench
openbiorq
os_harm
perfcodebench
```

planbench
proofaug
pxplore
real_webagents
realtimeqa
reliancescope
researchcodebench
revolve
risebench
robust_reasoning_benchmark
safeagentbench
sib200
software_world_models
sustainable_food
swe_smith
swebench_live
swepolybench
tau2_bench
taubench
truthfulqa_mc
tulu_human_eval
tumlu
visual_riddles
visualwebarena
vl_rewardbench
workarena
workbench_revisited

References

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025. URL https://arxiv.org/abs/2503.13335.

Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. Agent psychometrics: Task-level performance prediction in agentic coding benchmarks. In *Conference on Language Modeling*, 2026. URL https://arxiv.org/abs/2604.00594.

Sin-Han Yang, Cheng-Kuang Wu, Chieh-Yen Lin, Yun-Nung Chen, Hung-yi Lee, and Shao-Hua Sun. On calibration of large language models: From response to capability. *arXiv preprint arXiv:2602.13540*, 2026. URL https://arxiv.org/abs/2602.13540.

AIMS Foundations. measurement-db: Public benchmark inventory and response data. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/aims-foundations/measurement-db.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2002.10957.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL https://github.com/aims-foundations/paiec_baseline.